

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Java 全部 8 种基本类型,能说出与 C 的 `char/int/long/float/double` 的对应关系与差异
- 理解 JVM 与字节码:"一次编译、处处运行"的机制,以及"int 不是对象"的含义
- 掌握变量声明、初始化与默认值规则,并会使用 `var` 局部类型推导
- 掌握字面量写法(二进制/下划线/后缀/文本块)与 `final` 常量的语义(与 C 的 `const` 对比)
- 掌握类型转换规则,避开截断、溢出等常见坑;理解包装类与自动装箱的 `==` 陷阱
- 掌握字符串的不可变性与常用 API、数组的基础操作与类型安全的枚举

💡 从 C 到 Java:本章主题如何迁移

如果你会用 C 写 `int n = 0; char *s = "hi";`,那么 Java 的语法你几乎已经会了——但有三件"世界观"上的事必须先转换。**第一,程序不再直接跑在操作系统上,而是跑在 JVM 里。**C 编译器把源码编译成机器码交给 CPU;Java 编译器把源码编译成**字节码**,交给 JVM 解释/即时编译执行。代价是冷启动稍慢,收益是同一份字节码在 Windows、Linux、macOS 上行完全一致——这是生产环境"编译一次、处处运行"的基础。你的 `main` 是"JVM 调用的入口",所以签名固定为 `public static void main(String[] args)`。

第二,基本类型是"值",其他一切都是"引用"。C 里数组名是"退化的指针",Java 里数组是对象;C 里结构体可以整体拷贝,Java 里对象只能通过引用操作。`int`、`double` 等 8 种基本类型像 C 的值类型,其余一切(数组、字符串、对象)都是引用——引用类似 C 的指针,但没有指针运算、不会越界、不可能悬垂。

第三,变量有"默认值",内存由 JVM 管。C 的局部变量不初始化是未定义行为;Java 规定成员变量自动获得零值,局部变量则必须先初始化才能用,否则编译直接报错——把"未定义行为"变成"编译期错误"。`malloc/free` 彻底消失,取而代之的是自动垃圾回收(第5章详解)。

📦 前置知识自查

- C 基本类型的字节数与范围(`sizeof`);变量声明与初始化;局部变量与全局变量(`static`)的区别
- C 的整型字面量(0x 十六进制、0 八进制)与转义序列(`\n`、`\t`);隐式转换(整型提升)与强制转换(截断)
- C 的字符串(`char[]/char*`)与数组操作(`strlen/strcpy`、下标访问)
- C 的 `enum` 枚举与 `#define` 宏常量的局限

🚀 本章学习路线

1. **第一阶段(约 1.5 小时,建立世界观):**通读 §1.1–§1.3,理解 JVM、8 种基本类型与变量规则,对照"C 写法 | Java 写法"表格体会差异
2. **第二阶段(约 1.5 小时,动手写):**运行 §1.4–§1.8 的示例代码,重点实验 §1.6 的 `==` 陷阱与 §1.7 的字符串 API
3. **第三阶段(实验,约 1 小时):**独立完成章末实验"学生成绩登记与统计",用自测题验收
4. **验收标准:**不看资料能说出 8 种基本类型、成员变量与局部变量初始化的区别、包装类 `==` 陷阱的完整机制

本章知识导图

- §1.1 Java 平台与第一个程序(JVM / 字节码 / 编译运行流程 / printf)
- §1.2 基本类型总览(8 种类型表 / 与 C 一一对照 / boolean 与 char / 编码)
- §1.3 变量:声明、初始化、默认值与 var(成员变量零值 / 局部变量必须初始化)
- §1.4 字面量与常量(进制 / 下划线 / 后缀 / 文本块 / final 与 C 的 const)
- §1.5 类型转换(隐式拓宽 / 显式收窄 / 截断与除法丢精度)
- §1.6 包装类与自动装箱(装箱拆箱 / 缓存 / == 陷阱)
- §1.7 字符串与字符(不可变性 / API / 长度与字节 / == 与 equals)
- §1.8 数组入门(声明初始化 / length / 增强 for / 二维数组)
- §1.9 枚举:从 C 的 enum 到类型安全(带字段与方法 / values / valueOf)

§1.1 Java 平台与第一个程序

1.1.1 从"C 编译"到"Java 编译 + 运行"

C 的构建链是:源码 → 编译器 → 机器码 → 链接器 → 可执行文件,产物与 CPU、操作系统绑定。Java 的构建链是:**源码(.java) → javac 编译器 → 字节码(.class) → JVM 解释或即时编译(JIT)执行**。运行时代码路径是"先解释执行,热点代码被 JIT 编译成本地码"——所以 Java 程序有"预热"过程,性能测试必须考虑预热。

★ 重点:main 方法签名是固定的

入口方法必须精确写作 `public static void main(String[] args)`:`public` 让 JVM 能访问;`static` 使入口不依赖对象;`void` 表示返回给 JVM 的只有退出码;参数 `args` 对应 C 的 `argc/argv`。文件名必须与 `public` 类名一致。

示例 1-1 第一个 Java 程序(对照 C 的 hello.c)

```
// HelloWorld.java — 文件主名必须与 public 类名一致
public class HelloWorld {
    public static void main(String[] args) {
        // 编译: javac HelloWorld.java -> 生成 HelloWorld.class
        // 运行: java HelloWorld          (不带 .class 后缀)
        System.out.println("你好,Java!");
    }
}
```

⚠ 易错点 1:用 "java HelloWorld.class" 运行

`java` 命令的参数是 **类名** 而非文件名:应写 `java HelloWorld`。写了 `java HelloWorld.class` 会报 `Error: Could not find or load main class`。`javac` 要带 `.java` 后缀, `java` 不能带后缀——两者恰好相反。

1.1.2 输出与格式化:printf 的 Java 版本

Java 提供三层输出: `System.out.print` (不换行)、`System.out.println` (换行)、`System.out.printf` (格式化,语法与 C 的 `printf` 几乎相同)。**区别:Java 用 `%n` 表示跨平台换行;字符串拼接用 `+` 运算符(任意类型都能拼)。**

示例 1-2 printf 与字符串拼接

```

public class PrintDemo {
    public static void main(String[] args) {
        int n = 42;
        double pi = 3.1415926;
        String name = "张三";
        System.out.println("n = " + n);           // 拼接:任何类型 + 字符串
        System.out.printf("n = %d, pi = %.2f%n", n, pi); // 保留 2 位小数
        System.out.printf("姓名: %s,十六进制: %x,八进制: %o%n", name, n, n);
        System.out.printf("左对齐: %-8d| 补零: %05d%n", n, 7);
        System.out.printf("科学计数: %e%n", pi);
    }
}

```

§1.2 基本类型总览:与 C 一一对照

1.2.1 8 种基本类型

Java 只有 8 种基本类型,没有任何 unsigned 类型,也没有"int 大小由平台决定"这类 C 问题——所有类型的大小、范围由语言规范硬性规定。同样的 int 在 32 位与 64 位机器上都是 4 字节。

类型	字节	取值范围	C 对照
byte	1	-128 ~ 127	signed char
short	2	-32768 ~ 32767	short
int	4	约 ±21 亿	int(最常见)
long	8	约 ±9.2×10 ¹⁸	long long
float	4	约 ±3.4×10 ³⁸ (约 7 位有效数字)	float
double	8	约 ±1.8×10 ³⁰⁸ (约 15-16 位有效数字)	double
char	2	0 ~ 65535(UTF-16 码元)	char(1 字节,ASCII)
boolean	未规定	true / false	C 没有真正的 bool

三个关键差异:① **boolean 是独立类型**——C 用 0/非0 表示真假,Java 的 if/while 条件必须是 boolean,写 if (1) 直接编译错误,消灭了一大类"把赋值当比较"的 C 经典 bug;② **char 是 2 字节**——16 位无符号,存 UTF-16 编码单元,可直接写 '中';③ **没有 unsigned**——需要无符号语义时用更大的类型(如 long)或 Integer.toUnsignedString 等工具方法。

例题 1.1 检查你的 C 直觉:这行代码错在哪?

```

public class TypeDemo {
    public static void main(String[] args) {
        char ch = '中';           // 合法:char 是 16 位,容纳汉字
        boolean ok = 1;           // 编译错误:1 不能自动转成 boolean
        byte b = 128;             // 编译错误:128 超出 byte 范围(字面量默认 int)
        long big = 3000000000L;    // 合法:加 L 后缀才是 long 字面量
        System.out.println(ch + " " + big);
    }
}

```

1.2.2 字符与编码:从 ASCII 到 Unicode

C 时代一个字符一个字节,处理中文要靠 setlocale 与多字节编码"体操"。Java 出生就是 Unicode:char 是 UTF-16 编码单元,字符串内部是 UTF-16 序列,源码本身也允许中文标识符(但不建议)。生产要点:文件读写一律显式指定 UTF-8,避免 Windows 默认编码(GBK)导致的乱码——第7章详解。

§1.3 变量:声明、初始化与默认值

1.3.1 声明语法与 C 的差异

Java 声明变量与 C 几乎一致: 类型 名字;。但有两差异: ① 数组声明必须写作 `int[] a ( int a[]` 虽兼容但不建议); ② 没有全局变量——所有变量要么是类成员(静态/实例字段), 要么是局部变量, "全局"由类的 `public static` 字段承担。

★ 重点:默认值的完整规则

- **成员变量(字段):**自动获得零值——数值 0/0.0, boolean 为 false, char 为 `'\u0000'`, 引用类型为 `null`
- **局部变量:**没有任何默认值, 使用前必须显式初始化, 否则编译报错 "variable might not have been initialized"
- **数组元素:**如同成员变量, 自动获得零值(`new int[5]` 得到 5 个 0)

示例 1-3 默认值实验

```
public class DefaultValues {
    // 成员变量:自动零值
    static int i;
    static double d;
    static boolean flag;
    static String name;

    public static void main(String[] args) {
        System.out.println(i);    // 0
        System.out.println(d);    // 0.0
        System.out.println(flag); // false
        System.out.println(name); // null
        // 局部变量:必须先初始化才能读
        int x = 0;
        System.out.println(x);
        // int y; System.out.println(y); // 编译错误:可能未初始化
    }
}
```

1.3.2 var:局部变量类型推导(Java 10+)

C 没有类型推导; Java 10 引入 `var`, 让编译器从初始化表达式推断类型。 `var` 只适用于局部变量(含增强 for 循环变量、lambda 参数), 不能用于字段、方法参数与返回值——公共 API 必须显式标注类型, 这是工程约定。

示例 1-4 var 的使用边界

```
import java.util.ArrayList;

public class VarDemo {
    // var name = "x"; // 编译错误:var 不能用于字段
    public static void main(String[] args) {
        var count = 10;           // 推断为 int
        var name = "Java";        // 推断为 String
        var list = new ArrayList<String>(); // 推断为 ArrayList<String>
        for (var i = 0; i < 3; i++) { // 循环变量也可以用
            System.out.println(i);
        }
        // var x; // 编译错误:必须初始化
        // var y = null; // 编译错误:null 无法推断类型
        System.out.println(count + name + list.getClass());
    }
}
```

工程建议:什么时候用 var

生产惯例: 类型"又长又明显"时用 var(如 `var map = new HashMap<String, List<Integer>>()`),类型不直观或影响可读性时显式写出。公共 API 的签名永不使用 var。这是与 C 最大的风格差异之一,团队规范里通常会写明。

§1.4 字面量与常量

1.4.1 字面量写法:比 C 更丰富

Java 支持 C 的全部整型字面量风格(十进制、0x 十六进制、0 八进制),并新增**二进制字面量(0b)**与**下划线分隔符**。浮点字面量默认是 double,要 float 必须加 F 后缀;long 加 L 后缀(大写 L 避免与 1 混淆)。字符转义与 C 相同(`\n \t \\ \' \"`),另有 Unicode 转义 `\uXXXX`。

示例 1-5 字面量全集

```
public class LiteralDemo {
    public static void main(String[] args) {
        int hex = 0x1F;           // 十六进制:31
        int oct = 075;             // 八进制(前缀 0):61
        int bin = 0b1010;         // 二进制:10
        long big = 100_000_000_000L; // 下划线分隔,long 后缀 L
        double pi = 3.141_592_653; // 浮点下划线
        float f = 3.14F;          // float 必须加 F
        double sci = 1.5e3;        // 科学计数法:1500.0
        char c = '中';             // Unicode 转义:中
        char tab = '\t';           // 转义序列与 C 相同
        System.out.println(hex + " " + oct + " " + bin);
        System.out.println(big + " " + pi + " " + f + " " + sci);
    }
}
```

1.4.2 文本块(Java 15+):告别 "\n" 地狱

C 里写多行字符串要手动拼接 `"line1\n" "line2\n"`;Java 15 起可用**文本块**直接书写多行字符串,生产中最常见的用途是内嵌 SQL、JSON、HTML 模板。

示例 1-6 文本块

```
public class TextBlockDemo {
    public static void main(String[] args) {
        // 三个双引号开始,内容原样保留(无需 \n 转义)
        String sql = """
            SELECT id, name, score
            FROM student
            WHERE score > 60
            ORDER BY score DESC
            """;
        System.out.println(sql); // 文本块内无需 \n 转义,原样输出
    }
}
```

1.4.3 常量:final 与 C 的 const

C 的 `const int MAX = 100;` 是"只读变量",仍可能被指针绕过;Java 的 `final` 更严格: **final 变量一旦赋值,任何代码都不能再修改它**。约定俗成:类级常量写作 `public static final`,命名全大写加下划线。

示例 1-7 final 常量(对照:C 的 `#define MAX 100` 是宏、`const int n = 10` 是只读变量)

```

public class ConstDemo {
    // 类级常量:全大写 + 下划线
    public static final double PI = 3.14159265358979;
    private static final int MAX_RETRY = 3;

    public static void main(String[] args) {
        final int local = 100;
        // local = 200;    // 编译错误:final 变量不能重新赋值
        System.out.println(PI + " " + MAX_RETRY + " " + local);
        // final 引用:引用不可变,但指向的对象内容可变
        final StringBuilder sb = new StringBuilder("a");
        sb.append("b");    // 合法:对象内容可以变
        System.out.println(sb);
        // sb = new StringBuilder(); // 编译错误:引用不能重新指向
    }
}

```

⚠ 易错点 2:final 引用 ≠ 内容不可变

`final` 修饰引用类型时,保证的是"引用不能再指向别的对象",不保证对象内部状态不变。 `final StringBuilder sb` 照样能 `sb.append()`。真正不可变的是 `String`、`Integer` 这类"不可变类",与 `final` 是两回事。

§1.5 类型转换:隐式与显式

1.5.1 隐式转换:只有"拓宽"是自动的

与 C 一样,Java 的隐式转换遵循"小 → 大"拓宽方向: `byte` → `short` → `int` → `long` → `float` → `double`,以及 `char` → `int`。任何"大 → 小"的收窄都必须显式强转。与 C 不同,Java 规则统一、无实现差异,不会出现"同一段代码不同编译器行为不同"。

1.5.2 显式强转与截断

示例 1-8 转换的规则与截断

```

public class CastDemo {
    public static void main(String[] args) {
        // 隐式拓宽:自动完成
        int i = 100;
        long l = i;
        double d = l;

        // 显式收窄:必须强转,可能丢失信息
        long big = 3_000_000_000L;
        int n = (int) big;    // 超出 int 范围:结果被截断,不可预料
        double pi = 3.99;
        int p = (int) pi;    // 截断:3(向零取整,与 C 相同)
        char ch = (char) 97;    // 'a'
        System.out.println(n + " " + p + " " + ch);

        // 经典坑:两个 int 相除,小数被丢弃
        int a = 7, b = 2;
        double r = a / b;    // 3.0!先整数除法得 3,再转 double
        double r2 = (double) a / b;    // 3.5,先把 a 转 double 再除
        System.out.println(r + " " + r2);
    }
}

```

⚠ 易错点 3:除法丢精度与强转截断

两个经典事故:① `double r = 7 / 2;` 得到 3.0 而不是 3.5——整数除法先执行,小数在转 `double` 之前就丢了;正确写法是 `(double) 7 / 2`。② 大 `long` 强转成 `int` 不报错,而是按位截断出"看似合理实则错误"的数,生产上应在转换前做范围检查(如 `if (big > Integer.MAX_VALUE)`)。

🚀 工程建议:金额永远用整数或 BigDecimal

float/double 是二进制浮点, $0.1 + 0.2$ 不等于 0.3 。涉及金额、税率的计算一律不用浮点:用 long (以分为单位)或 BigDecimal。这与 C 的教训一致,Java 只是提供了更顺手的工具。

§1.6 包装类与自动装箱:== 陷阱

1.6.1 为什么需要包装类

基本类型不是对象——数组、集合、泛型(第10章)要求元素是对象。于是每个基本类型都有对应**包装类**:byte→Byte、short→Short、int→Integer、long→Long、float→Float、double→Double、char→Character、boolean→Boolean。Java 5 起编译器自动在两者间转换,称为**自动装箱/拆箱**。

示例 1-9 装箱、拆箱与 == 陷阱(必背)

```
public class BoxDemo {
    public static void main(String[] args) {
        // 自动装箱:Integer.valueOf(int) 的语法糖
        Integer a = 100;        // 等价于 Integer a = Integer.valueOf(100);
        Integer b = 100;
        System.out.println(a == b);    // true:100 在 -128~127 缓存区间内

        Integer c = 200;
        Integer d = 200;
        System.out.println(c == d);    // false!超出缓存区间,两个不同对象
        System.out.println(c.equals(d)); // true:比较值,永远的正确姿势

        // 自动拆箱:Integer 参与算术时自动转 int
        Integer x = 10;
        int y = x + 1;            // 拆箱后相加:11
        System.out.println(y);

        // 拆箱空指针陷阱
        Integer z = null;
        // int w = z;            // 运行时 NullPointerException:null 不能拆箱
    }
}
```

⚠ 易错点 4:Integer == 的缓存区间 -128~127

装箱时,JVM 对 -128~127 之间的 Integer 复用缓存对象,所以 $a == b$ 是 true;超过这个区间每次装箱都新建对象, $c == d$ 是 false。同一段代码,改个数值结果反转——这是生产中最隐蔽的 bug 来源之一。铁律:包装类比较一律用 equals。另外 == 比较 Integer 与 int 时会发生拆箱,按值比较,行为正常。

§1.7 字符串与字符

1.7.1 字符串不可变:与 C 的 char[] 对照

C 的字符串是 char[],内容随时可改;Java 的 String 是**不可变对象**:任何"修改"操作(拼接、替换、截取)都返回新对象,原字符串不变。好处:线程安全、可放心共享、可作 HashMap 的键;代价:循环里拼接必须用 StringBuilder,否则会产生大量临时对象。

C 的写法	Java 的写法
<code>char s[] = "hello" (可变)</code>	<code>String s = "hello" (不可变)</code>
<code>strlen(s) (字节数)</code>	<code>s.length() (UTF-16 码元数)</code>
<code>strcpy(dst, s)</code>	<code>s2 = s (引用赋值即可)</code>
<code>strcat(dst, s)</code>	<code>s1 + s 或 s1.concat(s)</code>
<code>strcmp(a, b) (按字节)</code>	<code>a.equals(b) (按值比较!)</code>
<code>sprintf(buf, "%d", n)</code>	<code>String.format("%d", n) 或 Integer.toString(n)</code>

示例 1-10 字符串常用 API

```
public class StringDemo {
    public static void main(String[] args) {
        String s = "Java 教程";
        System.out.println(s.length());           // 6(UTF-16 码元数,不是字节数)
        System.out.println(s.charAt(0));           // 'J'
        System.out.println(s.substring(0, 4));     // "Java"(左闭右开)
        System.out.println(s.indexOf("教程"));     // 5
        System.out.println(s.contains("va"));     // true
        System.out.println(s.replace("教程", "使用大全")); // 返回新串,原串不变

        // 切分与拼接
        String[] parts = "a,b,c".split(",");       // -> [a, b, c]
        String joined = String.join("-", parts);    // -> a-b-c

        // 频繁拼接:用 StringBuilder
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 1000; i++) {
            sb.append(i).append(',');
        }
        System.out.println(sb.length());           // 3890,而不是 1000 个新对象

        // 数值与字符串互转
        int n = Integer.parseInt("42");            // 字符串 -> 数值
        String s2 = Integer.toString(42);          // 数值 -> 字符串

        // == 与 equals:字符串常量池
        String t1 = "abc";
        String t2 = "abc";
        String t3 = new String("abc");
        System.out.println(t1 == t2);              // true(编译期常量入常量池)
        System.out.println(t1 == t3);              // false(不同对象)
        System.out.println(t1.equals(t3));         // true(按值比较,正确姿势)
    }
}
```

⚠ 易错点 5:字符串比较用 == 的后果

"abc" == "abc" 是 true(编译期常量进常量池复用),但 new String("abc") == "abc" 是 false, scanner.next() == "abc" 也几乎必然是 false——凡是运行时构造出来的字符串,== 都不可靠。生产铁律:字符串比较一律 equals ;忽略大小写用 equalsIgnoreCase。C 用 strcmp,Java 用 equals,就是把"按值比较"从函数提升为方法。

§1.8 数组入门

1.8.1 声明、创建与 length

Java 数组与 C 数组的核心差异:数组是对象。声明 `int[] a` 只是声明引用;必须 `new int[5]` 才分配空间,元素自动获得零值。访问越界时 JVM 抛出 `ArrayIndexOutOfBoundsException`,而不是 C 的"静默越界"未定义行为。数组有 `length` 属性

——C 没有,C 程序员必须自己记大小,这是 C 数组最大的痛点。

示例 1-11 数组基础

```
import java.util.Arrays;

public class ArrayDemo {
    public static void main(String[] args) {
        int[] scores = new int[5];           // 5 个 0
        int[] primes = {2, 3, 5, 7, 11};     // 初始化(仅声明时可用)
        int[] more = new int[]{1, 2, 3};     // 匿名数组,可随时用

        System.out.println(primes.length);   // 5:length 是属性,不是函数
        System.out.println(scores[0]);       // 0(默认零值)

        for (int p : primes) {               // 增强 for:C 里没有
            System.out.print(p + " ");
        }
        System.out.println();

        Arrays.sort(primes);                 // 排序(原地)
        Arrays.fill(scores, 60);             // 全部填 60
        int[] copy = Arrays.copyOf(primes, 10); // 扩容拷贝,多余补 0
        System.out.println(Arrays.toString(primes)); // 转字符串方便打印

        // 二维数组:数组的数组,每行长度可以不同
        int[][] matrix = new int[3][4];      // 3 行 4 列
        int[][] triangle = {{1}, {1, 1}, {1, 2, 1}}; // 三角阵(每行不等长)
        System.out.println(matrix.length + " 行," + matrix[0].length + " 列");
    }
}
```

⚠ 易错点 6:数组是引用,拷贝要用 `copyOf`

`int[] b = a;` 只是让两个引用指向同一块内存——修改 `b` 会改 `a`,这与 C 的“数组名是首元素指针”本质相似。真正拷贝用 `Arrays.copyOf(a, len)` 或 `a.clone()`。生产上把数组传给方法也要小心:方法内修改会直接影响原数组(传引用,见第4章)。

§1.9 枚举:从 C 的 `enum` 到类型安全

1.9.1 C 的 `enum` 之痛

C 的 `enum {RED, GREEN, BLUE};` 本质是整型常量:名字全局可见、可以与其他整数混用(`int x = RED;` 随意)、没有字符串对应、不能携带附加信息。Java 的 `enum` 是一个完整的类:每个枚举值是它的实例,类型安全(`Season.SPRING` 不能赋给 `int`),可以带字段、构造器与方法,天然支持 `switch`,还有 `values()` / `valueOf()` / `ordinal()` 内置工具。

示例 1-12 带字段与方法的枚举

```

public enum Season {
    // 枚举值:每个都是 Season 的一个实例,可带参数
    SPRING("春", 1), SUMMER("夏", 2), AUTUMN("秋", 3), WINTER("冬", 4);

    private final String name;    // 附加字段
    private final int index;

    Season(String name, int index) {    // 构造器(自动为每个枚举值调用)
        this.name = name;
        this.index = index;
    }

    public String getName() { return name; }
    public int getIndex() { return index; }

    public static void main(String[] args) {
        Season s = Season.SPRING;
        System.out.println(s.getName() + " " + s.getIndex()); // 春 1
        for (Season x : Season.values()) {    // values():全部枚举值
            System.out.println(x.name() + ":" + x.ordinal());
        }
        Season parsed = Season.valueOf("SUMMER"); // 字符串 -> 枚举
        System.out.println(parsed.getName());    // 夏
    }
}

```



工程建议:用枚举替代魔法数字

生产代码规范普遍要求: 状态、类型、等级等有限取值集合一律用 enum,禁止裸用 int 和字符串。好处:编译期检查(拼错常量名直接编译失败)、switch 穷尽性检查(第3章)、携带业务信息(名称、颜色、权重)。这是与 C 的 #define 宏枚举完全不同的体验。

本章速查表

主题	要点 / 写法
基本类型	byte short int long float double char boolean,共 8 种,大小平台无关
boolean	只接受 true/false,if 条件必须是 boolean(C 的 0/非0 失效)
char	16 位无符号,UTF-16 码元,可直接写 '中'
成员变量	自动零值:0 / 0.0 / false / '\u0000' / null
局部变量	必须初始化才能使用,否则编译错误
var	仅局部变量(Java 10+);字段/参数/返回值不可用;必须初始化
字面量	0b1010 二进制、1_000 下划线、100L、3.14F、1.5e3
文本块	""" 多行字符串 """(Java 15+),适合 SQL/JSON
常量	public static final 全大写加下划线;final 引用 ≠ 内容不可变
隐式转换	只允许拓宽:byte→short→int→long→float→double
显式转换	(int) x 收窄强转;截断向零取整;7/2 得 3,要 3.5 先转 double
包装类	Integer/Character 等;自动装箱拆箱;-128~127 有缓存,== 有坑
比较铁律	包装类、String 一律 equals;字符串常量池使 == 结果"看运气"
String 不可变	拼接产生新对象;循环拼接用 StringBuilder
字符串互转	Integer.parseInt("42") / Integer.toString(42) / String.valueOf(x)
数组	int[] a = new int[5];元素零值;length 属性;越界抛异常
数组工具	Arrays.sort / fill / copyOf / toString / binarySearch
枚举	完整类:带字段/方法;values() / valueOf() / ordinal();类型安全

最小必会清单

- 能默写 8 种基本类型及字节数,并指出与 C 的对应关系(含 long long、无 unsigned 的差异)
- 能解释 JVM 与字节码:为什么同一个 .class 文件可以跨平台运行
- 能说出成员变量与局部变量初始化规则的差异,以及数组元素的默认值
- 会写 var 并说出使用边界(仅局部变量、必须初始化、公共 API 不用)
- 会写二进制/下划线/后缀字面量与文本块,能处理其中的转义
- 能说清 final 与 C const 的异同,以及"final 引用 ≠ 内容不可变"
- 能完整复述 Integer 缓存陷阱:-128~127 与 == 的关系,并坚持用 equals
- 能写出字符串常用操作并解释 length() 是码元数、String 不可变
- 会声明数组、用 length、增强 for、Arrays 工具,并知道"等号只拷贝引用"
- 能写出带字段与方法的枚举,并用 values()/valueOf() 操作

自测题

Q 1. 下列代码哪些行会编译错误? `byte b = 128; long l = 3000000000L; boolean ok = 1; char c = '中'; double d = 3.14F;`

 参考答案

`byte b = 128` 编译错误(128 是 int 字面量,超出 byte 范围,且 int 不能隐式收窄为 byte); `boolean ok = 1` 编译错误(boolean 不接受整数)。其余合法:long 带 L 后缀合法;char 是 16 位可放汉字;float 字面量可以自动拓宽为 double。

Q 2. `int a = 7, b = 2; double r = a / b;` 结果是多少?如何得到 3.5?

✅ 参考答案

r 是 3.0:先做整数除法 $7/2=3$,再隐式转成 double。要 3.5 应写 `double r2 = (double) a / b;` (强转优先于除法),或 `a / 2.0`。

Q 3. 解释下面的输出,并给出修正: `Integer x = 200, y = 200; System.out.println(x == y);`

✅ 参考答案

输出 false。200 超出 Integer 缓存区间 -128~127,自动装箱各创建新对象,== 比较引用。改成 `x.equals(y)` 即 true。若两数是 127 则 == 为 true,同一段逻辑换数值结果反转,这正是它危险的原因。

Q 4. 结果分别是什么? `String a = "ab"; String b = "a" + "b"; String c = new String("ab");` (提示:考虑编译期常量折叠与常量池)

✅ 参考答案

`a == b` 是 true("a"+"b" 是编译期常量,折叠为 "ab" 入常量池); `a == c` 是 false(new 必出新对象)。生产结论:除了常量折叠的少数情况,字符串比较一律用 equals。

Q 5. 枚举 `Direction { UP, DOWN, LEFT, RIGHT }`,如何用字符串 "UP" 得到枚举值?如何遍历?

✅ 参考答案

`Direction.valueOf("UP")` 返回 UP(字符串不存在时抛 `IllegalArgumentException`);遍历用 `for (Direction d : Direction.values())`。`ordinal()` 返回声明序号,但生产上不要依赖 ordinal 做业务逻辑(枚举顺序调整会破坏它)。

Q 6. `final StringBuilder sb = new StringBuilder("a"); sb.append("b"); sb = new StringBuilder();` 哪一行编译错误?为什么?

✅ 参考答案

第三行 `sb = new StringBuilder()` 编译错误:final 保证引用不能重新指向,但对象内部状态可变,所以 `sb.append("b")` 合法。这就是"final 引用 ≠ 内容不可变"的现场演示。

章末实验题:学生成绩登记与统计

实验 1:学生成绩登记与统计

实验目标:编写一个控制台程序,登记学生信息并统计成绩,覆盖本章全部知识点。

实验要求:

- 任务 1:定义常量 MAX_STUDENTS 与及格线(覆盖 §1.4 常量/final)
- 任务 2:定义成绩等级枚举,带中文名(覆盖 §1.9 枚举)
- 任务 3:定义 Student 类,字段覆盖整型/浮点/布尔/字符/字符串/枚举(覆盖 §1.2 基本类型)
- 任务 4:控制台循环输入,输入 q 结束(覆盖 §1.3 变量、§1.7 parseInt 字符串互转)
- 任务 5:统计平均/最高/最低/各等级人数,格式化输出(覆盖 §1.1 printf、§1.8 数组)
- 任务 6:按成绩降序排序并打印成绩单(覆盖 §1.8 数组拷贝与排序)

实验环境:JDK 17+;命令: javac GradeBook.java 编译, java GradeBook 运行。

第一步 写"骨架":main + 常量 + 枚举 + Student 类,先编译通过再填逻辑

第二步 现输入循环:用 Scanner 读取, Integer.parseInt / Double.parseDouble 转换——输入非法字符会抛异常,先不管(第9章学异常后回来加固)

第三步 现统计与排序:复制数组用普通 for 循环;排序用 C 程序员最熟悉的选择排序,逐行翻译成 Java 语法

参考答案要点

```

import java.util.Scanner;

/** 学生成绩登记与统计 — 第1章综合实验参考答案 */
public class GradeBook {

    // 常量:final + static,全大写命名(对应 C 的 #define)
    public static final int MAX_STUDENTS = 100;
    private static final double PASS_LINE = 60.0;

    // 枚举:从 C 的整型枚举升级为带中文名的类型安全枚举
    enum Grade {
        EXCELLENT("优秀"), GOOD("良好"), PASS("及格"), FAIL("不及格");

        private final String text;

        Grade(String text) {
            this.text = text;
        }

        public String text() {
            return text;
        }
    }

    // 学生类:字段覆盖数值/布尔/字符/字符串/枚举全部类型
    static class Student {
        String name;        // 字符串:姓名
        int id;             // 整型:学号
        double score;       // 浮点:成绩
        boolean passed;     // 布尔:是否及格
        char gender;        // 字符:性别 M/F
        Grade grade;        // 枚举:等级

        Student(String name, int id, double score, char gender) {
            this.name = name;
            this.id = id;
            this.score = score;
            this.gender = gender;
            this.passed = score >= PASS_LINE;
            this.grade = toGrade(score);
        }

        static Grade toGrade(double s) {
            if (s >= 90) return Grade.EXCELLENT;
            if (s >= 75) return Grade.GOOD;
            if (s >= PASS_LINE) return Grade.PASS;
            return Grade.FAIL;
        }
    }

    public static void main(String[] args) {
        Student[] students = new Student[MAX_STUDENTS]; // 数组:默认 null
        int count = 0;
        Scanner in = new Scanner(System.in);

        System.out.println("=== 学生成绩登记(输入 q 结束) ===");
        while (count < MAX_STUDENTS) {
            System.out.print("姓名: ");
            String name = in.nextLine();
            if (name.equals("q")) { break; } // 字符串比较必须用 equals
            System.out.print("学号: ");
            int id = Integer.parseInt(in.nextLine()); // 字符串 -> int
            System.out.print("成绩: ");
            double score = Double.parseDouble(in.nextLine()); // 字符串 -> double
            System.out.print("性别(M/F): ");
            char gender = in.nextLine().charAt(0); // 取首字符
            students[count] = new Student(name, id, score, gender);
            count++;
        }
    }
}

```

```

    }
    in.close();

    // 统计:总分 / 平均 / 最高 / 最低 / 各等级人数
    double sum = 0, max = 0, min = 100;
    int[] gradeCount = new int[Grade.values().length]; // 默认全 0
    for (int i = 0; i < count; i++) {
        Student s = students[i];
        sum += s.score;
        max = Math.max(max, s.score);
        min = Math.min(min, s.score);
        gradeCount[s.grade.ordinal()]++; // 用序号做数组下标
    }
    double avg = (count == 0) ? 0.0 : sum / count;

    // 格式化输出:%n 是跨平台换行
    System.out.printf("=== 统计结果(共 %d 人) ===%n", count);
    System.out.printf("平均分: %.1f 最高: %.1f 最低: %.1f%n", avg, max, min);
    System.out.printf("优秀 %d 人 良好 %d 人 及格 %d 人 不及格 %d 人%n",
        gradeCount[0], gradeCount[1], gradeCount[2], gradeCount[3]);

    // 拷贝数组(等号只是引用拷贝,必须逐元素复制)
    Student[] sorted = new Student[count];
    for (int i = 0; i < count; i++) {
        sorted[i] = students[i];
    }
    // 选择排序:按成绩降序(与 C 完全一致的算法,只是换 Java 语法)
    for (int i = 0; i < count - 1; i++) {
        int k = i;
        for (int j = i + 1; j < count; j++) {
            if (sorted[j].score > sorted[k].score) {
                k = j;
            }
        }
        Student t = sorted[i];
        sorted[i] = sorted[k];
        sorted[k] = t;
    }

    System.out.println("=== 成绩单(降序) ===");
    for (int i = 0; i < count; i++) {
        Student s = sorted[i];
        System.out.printf("%2d. %-8s %c %6.1f %s%n",
            i + 1, s.name, s.gender, s.score, s.grade.text());
    }
}
}

```

设计说明:枚举 Grade 同时承载"等级判断"与"中文名"两类信息;passed 与 grade 由构造器统一推导,避免数据不一致;排序刻意用选择排序而非 Arrays.sort,便于与 C 实现逐行对照。扩展方向:用 HashMap 按姓名查学生、用 try-catch 处理非法输入(第9章)。

下一章预告:第2章 运算符与表达式。有了类型与变量,接下来就是"操作"——Java 的运算符与 C 表面相似,但溢出语义、除法、短路、== 比较等细节完全不同,处处是坑。